

Objetivo: realizar os cálculos para chegar ao salário líquido adequadamente.

Criar os arquivos:

SalarioView.xaml

SalarioViewModel.cs

Salario.cs

Incluir em: AppShell.xaml:

```
<ShellContent
    Title="Calculadora de Salario"
    ContentTemplate="{DataTemplate viewSalario:SalarioView}"
    Route="SalarioView" />
```

Incluir em SalarioView.xaml:

```
<StackLayout Margin="20">
    <Label Text="Calculadora de Salário" FontSize="Large" FontAttributes="Bold"
        Margin="0,0,0,10"/>
```

```
    <HorizontalStackLayout Spacing="10" Margin="0,0,0,10">
        <Label Text="Valor Hora:" VerticalOptions="Center" WidthRequest="130"/>
        <Entry Placeholder="0.00"
            Keyboard="Numeric"
            WidthRequest="140"
            Text="{Binding ValorHoraString, Mode=TwoWay}" />
    </HorizontalStackLayout>
```

```
    <HorizontalStackLayout Spacing="10" Margin="0,0,0,10">
        <Label Text="Horas Trabalhadas:" VerticalOptions="Center"
            WidthRequest="130"/>
        <Entry Placeholder="0"
            Keyboard="Numeric"
            WidthRequest="140"
            Text="{Binding HorasTrabalhadasString, Mode=TwoWay}" />
```

```
</HorizontalStackLayout>
```

```
<Button Text="Calcular" HorizontalOptions="Start"
        Command="{Binding CalcularSalarioCommand}"
        WidthRequest="150" Margin="0,10,0,10"/>

<Label Margin="0,10,0,0"
        FontAttributes="Bold" FontSize="Medium"
        Text="{Binding SalarioCalculado, StringFormat='Salário: {0:C}}'" />
</StackLayout>
```

Incluir em SalarioViewModel.cs:

```
public class SalarioViewModel : ObservableObject
{
    private readonly Salario _salario = new Salario();
    private decimal _salarioCalculado;

    public IRelayCommand CalcularSalarioCommand { get; }

    public SalarioViewModel()
    {
        CalcularSalarioCommand = new RelayCommand(
            execute: () => CalcularSalario(),
            canExecute: () => PodeCalcular()
        );
        // inicializações opcionais
        ValorHoraString = string.Empty;
        HorasTrabalhadasString = string.Empty;
    }
}
```

```

// Propriedades string usadas pelo Entry (TwoWay)

private string _valorHoraString;

public string ValorHoraString
{
    get => _valorHoraString;
    set
    {
        if (SetProperty(ref _valorHoraString, value))
        {
            // tenta atualizar o model (sem lançar)
            if (TryParseDecimal(value, out var parsed))
            {
                _salario.ValorHora = parsed;
            }
            else
            {
                _salario.ValorHora = 0m;
            }

            CalcularSalarioCommand.NotifyCanExecuteChanged();
            OnPropertyChanged(nameof(SalarioCalculado));
        }
    }
}

```

```

private string _horasTrabalhadasString;

public string HorasTrabalhadasString
{
    get => _horasTrabalhadasString;
    set
    {
        if (SetProperty(ref _horasTrabalhadasString, value))

```

```

    {
        if (TryParseDecimal(value, out var parsed))
        {
            _salario.HorasTrabalhadas = parsed;
        }
        else
        {
            _salario.HorasTrabalhadas = 0m;
        }
        CalcularSalarioCommand.NotifyCanExecuteChanged();
        OnPropertyChanged(nameof(SalarioCalculado));
    }
}

```

```

public decimal SalarioCalculado
{
    get => _salarioCalculado;
    private set => SetProperty(ref _salarioCalculado, value);
}

```

```

private void CalcularSalario()
{
    SalarioCalculado = _salario.CalcularSalario();
}

```

```

private bool PodeCalcular()
{
    return _salario.ValorHora > 0m && _salario.HorasTrabalhadas > 0m;
}

```

```

// tenta parsear considerando vírgula/ponto de acordo com cultura
private bool TryParseDecimal(string s, out decimal result)
{
    result = 0m;

    if (string.IsNullOrEmpty(s))
        return false;

    // tenta com CurrentCulture e InvariantCulture (aceita 1,23 e 1.23)
    if (decimal.TryParse(s, NumberStyles.Number, CultureInfo.CurrentCulture, out
result))
        return true;

    if (decimal.TryParse(s, NumberStyles.Number, CultureInfo.InvariantCulture, out
result))
        return true;

    // remover espaços e tentar novamente
    var cleaned = s.Trim();

    return decimal.TryParse(cleaned, NumberStyles.Number,
CultureInfo.CurrentCulture, out result)
        || decimal.TryParse(cleaned, NumberStyles.Number, CultureInfo.InvariantCulture,
out result);
}
}
}

```

Incluir em Salario.cs:

```

public class Salario
{
    // Propriedades usadas para o cálculo
    public decimal ValorHora { get; set; }

    public decimal HorasTrabalhadas { get; set; }
}

```

```
// Método chamado pelo ViewModel (CalcularSalario)
public decimal CalcularSalario()
{
    return ValorHora * HorasTrabalhadas * 10;
}
}
```